

GOOGLE AUTH

Integration & Deployment Guide: Google OAuth

For the backend staging and production servers to handle Google authentication successfully, you must configure the Google Cloud Console, update your Backend Environment Variables, and provide the correct information to the Frontend Developers.

Since the backend verify token flow accepts a pre-validated idToken from the frontend, the configuration setup is highly streamlined. Here is the step-by-step guide:

1. Developer Dashboard Setup (Google Cloud Console)

Go to the Google Cloud Console Credentials Page and log in to the account holding your Google Developer project.

A. Staging Environment Configuration

If you are using the same Google project for staging and production (or have a dedicated staging project), you must whitelist the frontend staging domains:

- Select your project and edit the OAuth 2.0 Client ID (Web application).

- Under Authorized JavaScript origins:

Add the frontend's staging URL (e.g.,

`https://trendupp-web.vercel.app/`, `https://app.trendupp.com/`, `http://app.trendupp.com` or whatever URL the frontend devs are deploying to).

- Under Authorized redirect URIs:

- If the frontend uses redirect-based sign-in, add the frontend callback URL (e.g.,

`https://trendupp-web.vercel.app/auth/google/callback`, or

`https://app.trendupp.com/auth/google/callback`).

- Note: The backend staging URL `https://trendupp-server.onrender.com/` does not need to be whitelisted here, as the backend only validates the token and does not host the login redirection page.

B. Production Environment Configuration

It is best practice to create a separate Google Cloud project (or at least a separate Client ID) for Production.

1.Create or open the Production Web Client ID.

2.Under Authorized JavaScript origins:

Add your live production frontend URL (e.g., `https://app.trendupp.com`).

3.Under Authorized redirect URIs:

Add the live frontend callback URL (e.g., `https://app.trendupp.com/auth/google/callback`).

2. Backend Environment Variables (.env)

You must configure the environment variables on both hosting platforms:

A. Staging (Render Dashboard)

Go to your service settings on Render, navigate to the Environment tab, and set/update:

```
GOOGLE_CLIENT_ID_WEB = [Your Web OAuth Client ID]
GOOGLE_CLIENT_ID_IOS = [Your iOS Client ID (if building a mobile app, otherwise leave blank or repeat web client ID)]
GOOGLE_CLIENT_ID_ANDROID = [Your Android Client ID (if building a mobile app, otherwise leave blank or repeat web client ID)]
GOOGLE_CLIENT_SECRET = [Your Google OAuth Client Secret]
```

B. Production (AWS Environment)

Depending on how you run the backend on AWS (EC2, ECS, or Elastic Beanstalk), set the same keys in your AWS environment/configuration dashboard:

```
GOOGLE_CLIENT_ID_WEB = [Your Production Web OAuth Client ID]
GOOGLE_CLIENT_ID_IOS = [Your Production iOS Client ID]
GOOGLE_CLIENT_ID_ANDROID = [Your Production Android Client ID]
GOOGLE_CLIENT_SECRET = [Your Production Google OAuth Client Secret]
```

3. What to Hand Over to Frontend Developers

Provide the frontend developers with the following details so they can perform the integration:

1.Google Web Client ID (GOOGLE_CLIENT_ID_WEB): They need this key to load the Google Sign-in button or SDK in their code.

2.API Endpoint URLs:

Staging Auth API: POST <https://trendupp-server.onrender.com/api/v1/auth/google>

Production Auth API: POST <http://16.171.252.107/api/v1/auth/google>

3.API Request Body Format:

```
json
{
  "idToken": "google_id_token_received_from_google_sdk",
  "role": "creator": "creator" or "brand"
  "acceptedTerms": true, // Optional: boolean
  "acceptedPromotions": true // Optional: boolean
}
```

4. What you need from the Frontend Developers

Their Hosting Base URL: You must ask them for their staging and production domain names. You will need to add these domains to the ALLOWED_ORIGINS environment variable in your backend config so that CORS does not block their API requests.

e.g., on Render update ALLOWED_ORIGINS to include their frontend staging URL (comma-separated).

 Start your free trial with \$300 in credit. Don't worry – you won't be charged if you run out of credit. [Learn more](#)

Google Cloud

TrendupTest

Search (/) for resources, docs, products and more

Google Auth Platform / Clients / Client: 29088640727-kth6lj5kjqur11njupb8423m0h5uqr45.apps.googleusercontent.com

Overview

Branding

Audience

Clients

Data access

Verification centre

Settings

Client ID for Web application

Delete

For use with requests from a browser

URIs 1 *

http://localhost:3001

URIs 2 *

https://trendupp-web.vercel.app

URIs 3 *

https://app.trendupp.com

+ Add URI

Client secrets

If you are in the process of cl

1

Viewing and down
a new one. Remen
never be checked

Client secret

Creation date

8 Jun

Status

✓ Er

+ Add secret

Authorised redirect URIs

For use with requests from a web server

URIs 1 *

http://localhost:3001/api/auth/callback/google

URIs 2 *

https://trendupp-web.vercel.app/api/auth/callback/google

URIs 3 *

https://app.trendupp.com/api/auth/callback/google

+ Add URI



TIKTOK AUTH

Integration & Deployment Guide: TikTok OAuth

Unlike Google, TikTok's V2 OAuth flow uses PKCE (Proof Key for Code Exchange) for security and requires a server-to-server token exchange. The frontend initiates the OAuth flow to get an authorization code and then sends that code, along with the PKCE verifier, to the NestJS backend to retrieve the login session.

Here is the step-by-step setup guide for TikTok Auth:

1. Developer Dashboard Setup (TikTok Developer Portal)

Go to the TikTok Developer Portal, log in, and open your Application.

Get Credentials:

Locate your Client Key and Client Secret. (Note that these will be used on the backend).

Whitelist Redirect URIs:

TikTok enforces strict validation: the redirect URI passed from the frontend to the backend must be exact and whitelisted in the developer portal.

Go to Redirect URIs in your TikTok app configuration and add the frontend redirect URLs:

For Staging: Add the frontend's staging callback URL (e.g.,
`https://trendupp-web.vercel.app/api/auth/callback/tiktok`).

For Production: Add the production frontend callback URL (e.g.,
`https://app.trendupp.com/api/auth/callback/tiktok`).

Configure Scopes:

Ensure your TikTok App has the user.info.basic scope enabled. Without this scope, the backend will fail to fetch the user's profile info (display name and avatar).

2. Backend Environment Variables (.env)

Configure the credentials in your backend environments:

A. Staging (Render Dashboard)

Go to your service settings on Render, and set/update:

TIKTOK_CLIENT_KEY = [Your TikTok App Client Key]

TIKTOK_CLIENT_SECRET = [Your TikTok App Client Secret]

B. Production (AWS Environment)

Set the environment variables in your AWS environment dashboard:

TIKTOK_CLIENT_KEY = [Your Production TikTok Client Key]

TIKTOK_CLIENT_SECRET = [Your Production TikTok Client Secret]

3. What to Hand Over to Frontend Developers

Provide the frontend developers with the following instructions:

Credentials: Share the TIKTOK_CLIENT_KEY with the frontend team.

PKCE Flow Requirement:

The frontend must generate a random Code Verifier (a cryptographically secure string between 43 and 128 characters).

They must generate a Code Challenge by hashing the Code Verifier using SHA-256 and converting it to hex format (as implemented and verified in the test client: `crypto.createHash('sha256').update(verifier).digest('hex')`).

TikTok Auth URL Redirection:

Redirect users to: <https://www.tiktok.com/v2/auth/authorize/>

Query Parameters:

client_key: [TikTok Client Key]

scope: user.info.basic

response_type: code

redirect_uri: The frontend callback URL (must match what is whitelisted on the portal)

state: The user role, e.g., "creator" or "brand"

code_challenge: [The generated Code Challenge]

code_challenge_method: S256

Backend Login API Endpoint:

When TikTok redirects back to the frontend with the code parameter, the frontend must send a POST request to the backend:

Staging Endpoint: <https://trendupp-server.onrender.com/api/v1/auth/tiktok>

Production Endpoint: <http://16.171.252.107/api/v1/auth/tiktok> or <http://app.trendupp.com/api/v1/auth/tiktok>

Request Body Format:

```
json
{
  "code": "auth_code_from_tiktok_redirect",
  "redirectUri": "the_exact_redirect_uri_used_in_authorization_step",
  "codeVerifier": "the_original_code_verifier_used_to_generate_challenge",
  "role": "creator", // Optional: "creator" or "brand"
  "acceptedTerms": true, // Optional: boolean
  "acceptedPromotions": true // Optional: boolean
}
```

Response: A successful login returns the NestJS JWT accessToken and user object.

4. What you need from the Frontend Developers

Their Redirect URIs: Ensure you collect the exact callback URIs they intend to use (for both staging and production) and verify that they are correctly added to the TikTok Developer Portal redirect list.

Confirm CORS Setup: Ensure their frontend domains are added to the backend's ALLOWED_ORIGINS environment variable.

INSTAGRAM AUTH

Integration & Deployment Guide: Instagram OAuth

Instagram OAuth (using the Instagram Graph API / Meta Developer Portal) requires the frontend to initiate authorization to obtain a temporary authorization code, which is then sent to the backend along with the redirect URI to be exchanged for a long-lived access token.

Here is the step-by-step setup guide for Instagram Auth:

1. Developer Dashboard Setup (Meta for Developers Portal)

Go to the Meta for Developers Console, log in, and select your App.

Verify API Access Product:

Make sure you have added the Instagram Graph API product (or Instagram Basic Display) to your App.

If accessing Creator or Business accounts (highly recommended for social commerce), ensure the App has the necessary business/login products configured.

Get Credentials:

Under App Settings > Basic, locate your App ID and App Secret. (These correspond to `INSTAGRAM_APP_ID` and `INSTAGRAM_APP_SECRET` on the backend).

Whitelist Redirect URIs:

In the Meta Developer Console, under the configuration settings for the Instagram product (or Facebook Login / Instagram Basic Display):

Locate the Valid OAuth Redirect URIs (or Client OAuth Settings).

Whitelist the frontend redirect URLs:

For Staging: Add the frontend's staging callback URL (e.g., <https://staging-app.trendupp.com/auth/instagram/callback>).

For Production: Add the production frontend callback URL (e.g., <https://app.trendupp.com/auth/instagram/callback>).

2. Backend Environment Variables (.env)

Configure the credentials in your backend environments:

A. Staging (Render Dashboard)

Go to your service settings on Render, and set/update:

`INSTAGRAM_APP_ID` = [Your Meta App ID]

`INSTAGRAM_APP_SECRET` = [Your Meta App Secret]

B. Production (AWS Environment)

Set the environment variables in your AWS environment dashboard:

`INSTAGRAM_APP_ID` = [Your Production Meta App ID]

`INSTAGRAM_APP_SECRET` = [Your Production Meta App Secret]

3. What to Hand Over to Frontend Developers

Provide the frontend developers with the following instructions:

Credentials: Share the INSTAGRAM_APP_ID with the frontend team.

Instagram Auth URL Redirection:

Redirect users to: <https://www.instagram.com/oauth/authorize>

Query Parameters:

client_id: [Instagram App ID]

redirect_uri: The frontend callback URL (must match what is whitelisted on the portal)

scope:

instagram_business_basic,instagram_business_manage_messages,instagram_business_manage_comments,instagram_business_content_publish,instagram_business_manage_insights (These allow reading basic profile info and managing insights/content)

response_type: code

state: The user role, e.g., "creator" or "brand"

Backend Login API Endpoint:

When Instagram redirects back to the frontend with the code parameter, the frontend must send a POST request to the backend:

Staging Endpoint: <https://trendupp-server.onrender.com/api/v1/auth/instagram>

Production Endpoint: <http://16.171.252.107/api/v1/auth/instagram>

Request Body Format:

```
json
{
  "code": "auth_code_from_instagram_redirect",
  "redirectUri": "the_exact_redirect_uri_used_in_authorization_step",
  "role": "creator", // Optional: "creator" or "brand"
  "acceptedTerms": true, // Optional: boolean
  "acceptedPromotions": true // Optional: boolean
}
```

Response: A successful login returns the NestJS JWT accessToken and user object.

4. What you need from the Frontend Developers

Their Redirect URIs: Ensure you collect the exact callback URIs they intend to use (for both staging and production) and verify that they are correctly added to the Meta for Developers portal redirect list.

Confirm CORS Setup: Ensure their frontend domains are added to the backend's ALLOWED_ORIGINS environment variable.